—----------cuenta.wlk

```
//CONTENIDOS.
class Contenido{
	const property titulo
	const property plan
	var property puntuacion = 0
	var property actores = #{}
	var property generos = #{}
	const property criterioDeValoracion
	const property esEstreno

	method valoracion() = criterioDeValoracion.calcularValoracion(self)

	method perteneceAGenero(genero) = generos.contains(genero)

	method seRelacionanEntreSi(perfil, otroContenido) = (self.valoracion() -
otroContenido.valoracion()).between(perfil.minDesvioPermitido(),
perfil.maxDesvioPermitido())
}

class Pelicula inherits Contenido{
	const property duracionMinutos
	var property minutosVistos = 0
	var property porcentajeVisto = 1

	method calcularPorcentajeDeVisualizacion(perfil){
		if(!perfil.puedeVer(self)){
			throw new ContenidoNoDisponibleException
			(message = "El contenido no está disponible en su plan actual.")
		}

		porcentajeVisto = (self.minutosVistos() / self.duracionMinutos()) * 100
	}

	method esMirable() = true

	method fueVistoPor(perfil) = perfil.contenidoVisto().contains(self)
}


class Serie inherits Contenido{
	const property temporadas = #{}
	var property cantidadDeTemporadas
   const property duracionCapitulos
	const property capitulosPorTemporada

   method ultimaTemporada() = temporadas.last()
```

```
    method agregarCapituloAUltimaTemporada(capitulo){
        self.ultimaTemporada().estrenar(capitulo)
    }

    method esMirable() = false

    method fueVistoPor(perfil) = temporadas.flatMap{
        temporada => temporada.capitulos()}.any{capitulo =>
perfil.contenidoVisto().contains(capitulo)}
}

object generosX12{
        method calcularValoracion(contenido) = contenido.generos().size() * 12
}

object temporadasXCapitulos{
    method calcularValoracion(serie) = serie.cantidadDeTemporadas() *
serie.capitulosPorTemporada()
}

object valoracion100{
    method calcularValoracion(serie) = 100
}

class Temporada{
        const property capitulos = #{}
        const property maximoDeCapitulos

        method cantidadDeCapitulos() = capitulos().size()

        method sePermiteOtroCapitulo() = cantidadDeCapitulos() + 1 <=
maximoDeCapitulos

        method estrenar(estreno){
            if(!self.sePermiteOtroCapitulo()){
                throw new ExcesoDeCapitulosException(message =
                "Se ha excedido el maximo de capitulos por temporada.")
            }

            capitulos.add(estreno)
        }

        method esMirable() = false

        method fueVistoPor(perfil) = perfil.contenidoVisto().any{capitulo =>
capitulos.contains(capitulo)}
}
```

```
class Capitulo{
	const property nombre
	const property duracionMinutos

	method esMirable() = true

	method fueVistoPor(perfil) = perfil.contenidoVisto().contains(self)

	method calcularPorcentajeDeVisualizacion(){
		porcentajeVisto = (self.minutosVistos() / self.duracionMinutos()) * 100
	}
}




//INSTANCIAS DE CONTENIDOS.
//Series.

const blackSails = new Serie(
	titulo = "Black Sails",
	generos = #{"Acción", "Drama", "Aventuras"},
	actores = #{"Toby Stephens", "Luke Arnold", "Toby Schmitz", "Hannah New", "Jessica
Parker Kennedy"},
	cantidadDeTemporadas = 4,
	capitulosPorTemporada = 8,
	duracionCapitulos = 48,
	criterioDeValoracion = temporadasXCapitulos,
	plan = planBasico,
	esEstreno = false)

const seanEternos = new Serie(
	titulo = "Sean Eternos",
	generos = #{"Fulbo"},
	actores = #{"Lionel Messi", "Angel Di Maria", "Xavi Hernández", "Luis Suarez"},
	cantidadDeTemporadas = 1,
	capitulosPorTemporada = 3,
	duracionCapitulos = 50,
	criterioDeValoracion = valoracion100,
	plan = planPremium,
	esEstreno = true)

const theWitcher =  new Serie(
	titulo = "The Witcher",
	generos = #{"Acción", "Fantasía", "Drama", "Misterio"},
	actores = #{"Henry Cavill", "Freya Allan", "Anya Chalotra"},
```

```
        cantidadDeTemporadas = 3,
        capitulosPorTemporada = 8,
        duracionCapitulos = 60,
        criterioDeValoracion = temporadasXCapitulos,
        plan = planBasico,
        esEstreno = true)

const goodOmens =  new Serie(
        titulo = "Good Omens",
        generos = #{"Fantasía", "Comedia", "Drama"},
        actores = #{"David Tennant", "Michael Sheen"},
        cantidadDeTemporadas = 6,
        capitulosPorTemporada = 6,
        duracionCapitulos = 50,
        criterioDeValoracion = temporadasXCapitulos,
        plan = planPremium,
        esEstreno = true)



//Películas.
const avengersEndgame = new Pelicula(
        titulo = "Avengers Endgame",
        duracionMinutos = 182,
        generos = #{"accion", "drama", "aventuras", "ciencia"},
        actores = #{"Robert Downey Jr", "Chris Evans", "Mark Ruffalo", "Chris Hemsworth",
        "Scarlett Johansson", "Jeremy", "Don Cheadle", "Paul Rudd", "Benedict
Cumberbatch", "Chadwick Boseman", "Brie Larson", "Tom Holland", "Karen Gillan", "Zoe
Saldana"},
        criterioDeValoracion = generosX12,
        plan = planBasico,
        esEstreno = true)

const it =  new Pelicula(
        titulo = "It",
        duracionMinutos = 135,
        generos = #{"terror"},
        actores = #{"Jaeden Martell", "Jeremy Ray Taylor", "Sophia Lillis", "Finn Wolfhard",
"Chosen Jacobs"},
        criterioDeValoracion = generosX12,
        plan = planPremium,
        esEstreno = true)

const eternoResplandor = new Pelicula(
        titulo = "El eterno resplandor de una mente sin recuerdos",
        duracionMinutos = 108,
        generos = #{"Drama", "Romance", "Ciencia Ficción"},
         actores = #{"Jim Carrey", "Kate Winslet", "Gerry Robert Byrne", "Elijah Wood"},
```

```
        criterioDeValoracion = generosX12,
        plan = planBasico,
        esEstreno = true)

const elementos = new Pelicula(
        titulo = "Elementos",
        duracionMinutos = 101,
        generos = #{"Animación", "Comedia", "Fantasía"},
        actores = #{"Leah Lewis", "Mamoudou Athie", "Ronnie Del Carmen", "Shila Ommi"},
        criterioDeValoracion = generosX12,
        plan = planBasico,
        esEstreno = true)

const milenium2 = new Pelicula(
        titulo = "Millennium 2: La chica que soñaba con una cerilla y un bidón de gasolina",
        duracionMinutos = 129,
        generos = #{"Accin", "Crimen", "Drama", "Misterio", "Suspenso"},
        actores = #{"Michael Nyqvist", "Noomi Rapace", "Lena Endre"},
        criterioDeValoracion = generosX12,
        plan = planPremium,
        esEstreno = true)



//Repositorio
const repositorio = #{avengersEndgame, it, eternoResplandor, elementos,
        milenium2, blackSails, seanEternos, theWitcher, goodOmens}


//PLANES Y CUENTAS.
//Planes.
object planBasico{
        method puedeMirar(contenido) = contenido.plan().equals(self)
}

object planPremium{
        method puedeMirar(contenido) = true
}



//Perfiles.
class Perfil{
        var property nombrePerfil
        const property plan = null
        var recomendador
        var property contenidoVisto =  #{}
        var property preferencias = #{}
```

```
	var property actoresFavoritos = #{}
	var property desvio = 0.15

	method agregarPreferencia(nuevaPreferencia) = preferencias.add(nuevaPreferencia)

	method agregarActorFavorito(nuevoActorFavorito) =
actoresFavoritos.add(nuevoActorFavorito)

	method promedioDeValoraciones(){
		if(self.contenidoVisto().size() > 0){
			const largoLista = self.contenidoVisto().size()
			const sumaDeValoraciones = self.contenidoVisto().map({elem =>
elem.valoracion()}).sum()

			return sumaDeValoraciones / largoLista
		}
		else{
			return 0
		}
	}

	method maxDesvioPermitido() = self.promedioDeValoraciones() +
self.promedioDeValoraciones()*self.desvio()

	method minDesvioPermitido() = self.promedioDeValoraciones() -
self.promedioDeValoraciones()*self.desvio()

	method puedeVer(contenido) = plan.puedeMirar(contenido)

	method ver(contenido, minutosVistos){
		if(!contenido.esMirable()){
			throw new ContenidoNoMirableException
			(message = "El contenido que intenta ver no es mirable")
		}

		contenido.minutosVistos(minutosVistos)
		contenidoVisto.add(contenido)
	}

	method vio(contenido) = contenido.fueVistoPor(self)

	method recomendarContenido(contenidoRecomendado) =
recomendador.esRecomendable(self, contenidoRecomendado)

	  && !self.vio(contenidoRecomendado)

	method esVariado() = recomendador.tienePreferenciasVariadas(self)
```

```
	method generarRecomendaciones() = repositorio.filter{
	recomendacion => self.recomendarContenido(recomendacion)}

	method leGustoBastanteUnContenido() = contenidoVisto.any{contenido =>
contenido.puntuacion().between(4,5)}

	method puntajePermitido(puntaje) = puntaje.between(1,5)

	method puntuarContenido(contenido, puntaje){
		if(!self.vio(contenido)){
			throw new ContenidoNoVistoException
			(message = "Para puntuar este contenido, primero debe verlo.")
		}

		if(!self.puntajePermitido(puntaje)){
			throw new PuntajeNoPermitidoException
			(message = "El puntaje debe ser del 1 al 5.")
		}

		contenido.puntuacion(puntaje)
	}

	method edadActual() = calendarioActual.hoy().year() -  fechaDeNacimiento.year()

	method cambiarRecomendador(nuevoRecomendador){
		if(self.edadActual() <= 13 ){
			throw new NoSePuedeCambiarRecomendadorException
			(message = "No podes cambiar de recomendador por ser menor de
edad.")
		}

		recomendador = nuevoRecomendador
	}
}

object calendarioActual{
	method hoy() = new Date()
}

const margo = margoZavala.crearPerfil("margo", valoracionSimilar)
const cosme = margoZavala.crearPerfil("cosme", preferenciaDeGenero)



//Cuentas.
class Cuenta{
	const property planCuenta
	var property perfiles = #{}
```

```
	method crearPerfil(nuevoNombre, nuevoRecomendador,
nuevaFechaDeNacimiento){
		const nuevoPerfil = new Perfil(nombrePerfil = nuevoNombre, plan =
planCuenta,
					recomendador = nuevoRecomendador,
fechaDeNacimiento = nuevaFechaDeNacimiento)

		if(nuevoPerfil.esMenorDe13()){
			nuevoPerfil.recomendador(recomendadorInfantil)
		}

		perfiles.add(nuevoPerfil)

		return nuevoPerfil
	}


	method crearPerfil(nuevoNombre, nuevaFechaDeNacimiento){
		const nuevoPerfil = new Perfil(nombrePerfil = nuevoNombre,
					plan = planCuenta, fechaDeNacimiento =
nuevaFechaDeNacimiento)

		if(nuevoPerfil.esMenorDe13()){
			nuevoPerfil.recomendador(recomendadorInfantil)
		}

		perfiles.add(nuevoPerfil)

		return nuevoPerfil
	}

	method esContenidoIdeal(contenido) = perfiles.all{
		perfil => perfil.recomendarContenido(contenido)}
}

const margoZavala = new Cuenta(planCuenta = planPremium, perfiles = #{})



//Recomendadores
object valoracionSimilar{
	method esRecomendable(perfil, contenidoRecomendado) =
		contenidoRecomendado.valoracion().between(perfil.minDesvioPermitido(),
perfil.maxDesvioPermitido())
		or perfil.contenidoVistos().isEmpty()

	method tienePreferenciasVariadas(perfil) = false
```

```
}


object preferenciaDeGenero{
	method esRecomendable(perfil, contenidoRecomendado) = perfil.preferencias().any{
	preferencia => contenidoRecomendado.perteneceAGenero(preferencia)}

   method tienePreferenciasVariadas(perfil) = perfil.preferencias().size() >= 5
}


object modoFan{
	method esRecomendable(perfil, contenidoRecomendado) =
perfil.actoresFavoritos().any{
		actor => contenidoRecomendado.actores().contains(actor)}

	method tienePreferenciasVariadas(perfil) = true
}

object recomendadorPorDefecto{
	method esRecomendable(perfil, contenidoRecomendado){
		if(contenidoRecomendado.puntuacion()>0){
			contenidoRecomendado.puntuacion().between(4,5)
		}
		else{
			contenidoRecomendado.esEstreno()
		 }
	}

	method tienePreferenciasVariadas(perfil) = true
}

object controlParental{
	method esRecomendable(perfil, contenidoRecomendado) =
contenidoRecomendado.perteneceAGenero("Infantil")

	method  tienePreferenciasVariadas(perfil) = true
}




//EXCEPCIONES
class ContenidoNoDisponibleException inherits Exception{}
class ExcesoDeCapitulosException inherits Exception{}
class ContenidoNoVistoException inherits Exception{}
```

```
class PuntajeNoPermitidoException inherits Exception{}
class ContenidoNoMirableException inherits Exception{}
class NoSePuedeCambiarRecomendadorException inherits Exception{}

—----------------test.wtest

import cuenta.*

describe "tests de cosmeFulanito"{

	test "cosmeFulanito pude ver blackSails" {
		assert.that(cosme.puedeMirar(blackSails))
	}

	//Me tira "Value was not false" porque en la primera entrega cosme no podía verlo, y
ahora sí.
	test "cosmeFulanito pude ver seanEternos" {
		assert.notThat(cosme.puedeMirar(seanEternos))
	}

	test "Calcular la valoracion de cosmeFulanito" {
		assert.equals(0, cosme.promedioDeValoraciones())
	}
}


describe "test de margoZavala"{

	test "margoZavala pude ver seanEternos"{
		assert.that(margo.puedeMirar(seanEternos))
	}

	test "margoZavala pude ver  blackSails"{
		margo.ver(blackSails)
		assert.that(margo.contenidoVistos().contains(blackSails))
	}

	test "Calcular la valoracion de margoZavala luego de ver avengersEndgame y
blackSails"{
		margo.ver(blackSails)
		margo.ver(avengersEndgame)
		assert.equals(34, margo.promedioDeValoraciones())
	}
}


describe "test Recomendaciones de margo"{
```

```
	test "margo podria ver seanEternos"{
		assert.that(margo.recomendarContenido(seanEternos))
	}

	test "margo luego de ver seanEternos, podria ver blackSails"{
		margo.ver(seanEternos)

		assert.notThat(margo.recomendarContenido(blackSails))
	}

	test "margo luego de ver seanEternos podria ver blackSails cambiando desvio a 70
%"{
		margo.ver(seanEternos)
		margo.desvio(0.7)
		assert.that(margo.recomendarContenido(blackSails))
	}
}


describe "test Recomendaciones de cosme"{

	test "cosme veria blackSails"{
		assert.that(cosme.recomendarContenido(blackSails))
	}

	test "cosme veria seanEternos"{
		assert.notThat(cosme.recomendarContenido(seanEternos))
	}

	test "cosme con agregado de preferencia de genero documental podria ver
seanEternos"{
		cosme.agregarPreferencia("documental")
		assert.that(cosme.recomendarContenido(seanEternos))
	}
}

describe "test cuentas y perfiles"{
	const elBarto = margoZavala.crearPerfil("elBarto", preferenciaDeGenero, 2000)
	const murphy = new Perfil(nombrePerfil = "murphy encias sangrantes",
recomendador = modoFan)

	method initialize(){
		elBarto.agregarPreferencia("animacion")
		elBarto.agregarPreferencia("comedia")
		elBarto.agregarPreferencia("accion")
		elBarto.agregarPreferencia("drama")
		elBarto.agregarPreferencia("fantasia")
		elBarto.agregarPreferencia("terror")
```

```
		murphy.agregarActorFavorito("jaeden")
		murphy.agregarActorFavorito("jeremy")
	}

	test "se añade El Barto y la cuenta de margo queda con 3 perfiles"{
		assert.that(margoZavala.perfiles().size() == 3)
	}

	test "el perfil de margo NO es variado"{
		assert.notThat(margo.esVariado())
	}

	test "el perfil de cosme NO es variado"{
		assert.notThat(margo.esVariado())
	}

	test "el perfil de El Barto es variado"{
		assert.that(elBarto.esVariado())
	}

	test "el perfil de Murphy es variado"{
		assert.that(murphy.esVariado())
	}

	test "Murphy, con recomendador modo fan, puede ver It"{
		assert.that(murphy.recomendarContenido(it))
	}
}

describe "test porcentaje visto de un contenido"{

	test "se vieron 90 minutos de los 182 que dura avengersEndgame"{
		margo.ver(avengersEndgame)
		avengersEndgame.minutosVistos(90)
		avengersEndgame.calcularPorcentajeDeVisualizacion(margo)
		assert.that(avengersEndgame.porcentajeVisto == 49.451) // 90/182*100 =
49.451
	}
}



describe "test disponibilidad de contenido según el plan"{

	method initialize(){
		const cuentaBasica = new Cuenta(planCuenta = planBasico, perfiles = #{})
		const pepe = cuentaBasica.crearPerfil("pepe", valoracionSimilar)
```

```
	}

	test "con planBasico milenium2 no esta disponible para ver" {
		assert.throwsExceptionWithMessage("El contenido no está disponible en su
plan actual.",
	}
}

describe "test puntuar un contenido"{
	test "si el contenido ya fue visto se puede puntuar"{
		margo.ver(avengersEndgame)
		margo.puntuarContenido(avengersEndgame, 4)
		assert.that(avengersEndgame.puntuacion() == 4)
	}

	test "si el contenido no fue visto no se puede puntuar" {
		assert.throwsExceptionWithMessage("No tenes visto este contenido",
			{margo.puntuarContenido(it, 3)})
	}

	test "el puntaje que se le quiere dar a un contenido visto debe ser entre 1 y 5"{
		margo.ver(it)
		assert.throwsExceptionWithMessage("El puntaje debe ser entre 1 y 5",
			{margo.puntuarContenido(it, 6)})
	}
}


describe "test crear perfil sin recomendador"{
	const jorgito = margoZavala.crearPerfil("jorgito", 1999)

	test "jorgito utiliza el recomendador por defecto"{
		assert.that(jorgito.recomendador() == recomedadorPorDefecto)
	}
}


describe "test crear perfil infantil"{
	const lilMarto = margoZavala.crearPerfil("lil marto", 2013)

	test "lilMarto es menor, utiliza el recomendador infantil"{
		assert.that(lilMarto.recomendador() == controlParental)
	}
}
```